

/home/wkk/pq_ac/lattice-anonymous-credentials 里面现在的代码有匿名凭证所对应代码和累加器所对应代码，我们的方案是面向无人机场景的，是一个基于累加器的可撤销匿名凭证方案。共有无人机 UAV，可信机构 TA 和地面控制台 GCS 三个实体，其中 UAV 作为用户向 TA 申请匿名凭证，TA 运行匿名凭证相关代码和累加器相关代码向 UAV 发送相关数据，让 UAV 能够生成一个合法的且没被撤销的匿名凭证（匿名凭证本质是一个零知识证明）。之后如果 TA 想要撤销对应 UAV，可以在累加器中运行 Delete，使得对应的 UAV 匿名凭证失效。注意根据每个实体所做的事情，将代码进行划分。

第一阶段：初始化

TA 运行算法 Setup, KeyGen 和 AccGen，生成各类参数以及公私钥对 (PK_{TA}, SK_{TA}) ，将公钥放入公共参数 pp，并初始化累加器和 BML

/*其中，Setup 生成各种基础参数，KeyGen 生成颁发时的陷门采样所用的公私钥对，AccGen 生成累加器陷门采样所用参数和初始化累加器值，字典 DX 和状态 st_a 等，BML 是一个可以存储 20 个最近广播的列表，用来帮助离线无人机同步见证。这些其实代码已经有了，但是现在变成多实体运行，需要单独拎出来*/

AccGen(pp):

(1) $Compute(A, R_A) \leftarrow TrapGen(1^n, 1^m, q)$ $\square$ In fact, just getting R_A

(2) $acc_0 \leftarrow R_q^{(n)}$

(3) $k_a \leftarrow (0, 1^\lambda)$

(4) Initialize an empty dictionary $DX: M \rightarrow Add, Del, \perp \times N$ and a counter $ctr := 0$.

(5) Set $\mathcal{A}_0 := acc_0$, and $st_a := (k_a, ctr, acc_0, DX)$.

Output: $(R_A, \mathcal{A}_0, st_a)$.

TA 向无人机 UAV_i 与 GCS 发送 pp 和 PK_{TA}

第二阶段：注册

无人机 UAV_i 运行 UKeyGen，生成公私钥对 (PK_{UAV}, SK_{UAV}) ，并把 PK_{UAV} 和真实身份 ID_{UAV} 发给 TA 进行注册。//每个无人机有个 64 位序列号作为真实身份。

Algorithm 6.2: UKeyGen

Input: Public parameters pp as in Algorithm 3.1.

(1) $s \leftarrow U(T_1^{m_s})$.

▷ $m_s = 2d$

(2) $t \leftarrow D_s s \bmod qR$.

Output: $(upk, usk) = (t, s)$.

TA 收到后，生成该无人机 UAV_i 的 32 位撤销句柄 $x = H(PK_{UAV}, ID_{UAV})$ ，并存储元组 $\langle PK_{UAV}, ID_{UAV}, x \rangle$

GCS 运行算法 GKeyGen（和 UKeyGen 类似），生成公私钥对 (PK_{GCS}, SK_{GCS}) ，并把 PK_{GCS} 和 64 位真实身份 ID_{GCS} 发给 TA 进行注册

TA 收到后，TA 创建证书 $Cert_{GCS} = \langle PK_{GCS}, T_{GCS}, \sigma_{TA} \rangle$ ，其中 $\sigma_{TA} = Sign_{SK_{TA}}(PK_{GCS} \parallel T_{GCS})$ // T_{GCS} 是时间戳

并存储 $(ID_{GCS}, Cert_{GCS})$ 在 TA 本地数据库，然后 TA 将 $Cert_{GCS}$ 发给 GCS

第三阶段：匿名凭证颁发

无人机 UAV_i 准备获取合法匿名凭证

UAV:

$r \leftarrow U(T_1^{2d})$.

$$c = Ar + [D_x | D_s | D] \begin{bmatrix} x \\ s \\ m \end{bmatrix} \bmod qR \quad // \text{与原来相比，多了 } D_x \text{ 与 } x$$

UAV_i 发送 c 给 TA

TA 收到 c 之后，UAV 与 TA 运行非交互式零知识证明：// 已有代码

在零知识证明里，无人机证明它知道一组短向量 (r, s) 满足 $c - Dm = Ar + D_s s + D_x x \bmod qR$ ，并且 $D_s s = PK_{UAV} \bmod qR$ 。如果 TA 验证不通过，协议终止。

TA: // 采样得到签名，已有代码

$$v_3 \leftarrow D_{R^k, s_2}$$

$$t \leftarrow F(st)$$

$v' \leftarrow TSampler(R_1, R_2, A, u + c - v_3 a_3 \bmod qR, t, s_1, s_2, s_3, s_4, s_L, s_H).$

$Parse\ v' = [v'_{1,1} \parallel v'_{1,2} \parallel v_2^T]^T$

$sig' = (t, v'_{1,2}, v_2, v_3)$

$st \leftarrow st + 1$

TA 运行 Add, 生成 w_x

TA 将 (sig', w_x) 发给无人机 UAV_i

无人机 UAV_i 收到 (sig', w_x) 后

UAV:

$Parseras[r_{1,1}^T \parallel r_{1,2}^T]^T$ with $r_{1,i} \in R^d$

$v_{1,2} \leftarrow v'_{1,2} - r_{1,2}$

if $Verify'(pk; m; (t, v_{1,2}, v_2, v_3); pp) = 0$, return $\perp$

if $MemVer(A, B_a, u_a, \mathcal{A}, x, w_x) = 0$, return $\perp$

去盲后, 验证签名和见证是否正确

无人机 UAV_i 拿到 (sig, w_x) 运行零知识证明生成协议, 生成匿名凭证 AC。也就是证明原有约束成立, 累加器线性约束和各个向量足够短。/*这一步零知识证明我后续会单独用专门的库生成, 做不出来先跳过。*/

第四阶段：认证与撤销

无人机到达某一 GCS 附近后, 验证 GCS 的证书, 验证通过后向 GCS 出示匿名凭证 AC, 并由 GCS 进行验证, 验证通过后即可提供服务, 如提供附近空间信息和参数。

当 TA 确定任务取消或结束时, 又或者确定某个撤销句柄对应的无人机是恶意的時候,

TA 找到对应无人机的撤销句柄 y 运行 Delete 算法，删除撤销句柄 y 对应的无人机

TA:

$(\mathcal{A}', upmsg, st'_a) \leftarrow Delete(A, B_a, u_a, R_A, st_a, \mathcal{A}, y, w_y) \boxtimes Algorithm2.2$

向所有无人机广播更新消息 $upmsg$

无人机 UAV_i 收到 $upmsg$ 后，运行 $MenWitUp$ 算法同步自身见证

UAV:

$w'_x \leftarrow MenWitUp(A, B_a, u_a, x, s_x, upmsg)$

if $MemVer(A, B_a, u_a, \mathcal{A}, x, w'_x) = 0$ 或者离线错过了一些删除广播

无人机通过安全信道向 TA 发送 $(x, upmsg)$

TA 收到 $(x, upmsg)$ 后

TA:

if 在 BML 中检索到 $upmsg$

TA 向 UAV_i 补发最新广播消息 $upmsg'$ 组

无人机 UAV_i 收到 $upmsg'$ 组后，运行 $MenWitUp$ 算法更新本地见证

Else

$w''_x \leftarrow MenWitSync(A, B_a, u_a, R_A, st_a, \mathcal{A}, x)$

发送 w''_x 给 UAV_i

无人机 UAV_i 收到 w''_x 后，更新本地见证为 w''_x

else

更新本地见证为 w'_x